

Задача 1: Аудит Уникальных Артикулов (Уникальные элементы)

Вы - аудитор склада. У вас есть полный список артикулов всех полученных товаров. Ваша задача — определить новинки: те артикулы, которые были получены ровно один раз (без дубликатов), чтобы выделить их из общей массы. Порядок следования новинок должен быть сохранен.

Требуется: Написать программу, которая анализирует список `inventory_ids` и формирует новый список, включающий только те элементы, которые встречаются в исходном списке единожды.

Исходные данные (Пример):

```
inventory_ids = [101, 205, 101, 303, 400, 205, 501, 303, 600]
```

Технические Шаги:

Нужно пройти по списку дважды:

1. Первый проход (Подсчет): Определить, сколько раз встречается каждый элемент. Для этого нужно будет использовать список (`counts`) для подсчета частоты (`frequency map`).
2. Второй проход (Фильтрация): Создать новый список (`unique_items`), проходя по исходному списку и добавляя элемент, только если его счетчик в словаре равен 1.

Пример Вывода:

```
--- Аудит Склада: Новинки ---  
Исходный список артикулов: [101, 205, 101, 303, 400, 205,  
501, 303, 600]  
Частота встречаемости: 101: 2, 205: 2, 303: 2, 400: 1,  
501: 1, 600: 1  
Список ID новинок (уникальные): [400, 501, 600]
```

Задача 2: Сортировка данных по коду региона (Сортировка по последней цифре)

Вы аналитик в логистической компании. У вас есть список кодов партий товаров. В вашей системе последняя цифра кода всегда обозначает регион доставки. Вам нужно сгруппировать все партии так, чтобы они следовали по возрастанию региона доставки.

Требуется: Написать программу, которая сортирует список целых чисел (`package_codes`) на основе последней цифры каждого числа.

Исходные данные (Пример):

```
package_codes = [435, 122, 581, 794, 615, 303, 221, 102]
```

Ожидаемый результат (Пояснение):

| Исходное число | Последняя цифра (Ключ) |

| 435 | 5 |

| 122 | 2 |

| 581 | 1 |

| 794 | 4 |

| 615 | 5 |

| 303 | 3 |

| 221 | 1 |

| 102 | 2 |

Сортировка по ключу (1, 1, 2, 2, 3, 4, 5, 5) приведет к следующему порядку:

```
[581, 221, 122, 102, 303, 794, 435, 615]
```

Пример Вывода:

```
--- Логистика: Сортировка по Региону ---
```

```
Исходные коды партий: [435, 122, 581, 794, 615, 303, 221, 102]
```

```
Отсортированные коды по региону (последняя цифра): [581, 221, 122,  
102, 303, 794, 435, 615]
```

Задача 3: Аудит Соответствия Стандартам (Подсчёт элементов, кратных числу)

Вы — инженер по контролю качества. У вас есть список результатов измерений. Согласно стандарту, только те измерения, которые кратны определенному калибровочному числу N (задается пользователем), считаются прошедшими первичную проверку. Вам нужно посчитать, сколько измерений соответствует стандарту.

Требуется:

Написать программу, которая:

1. Принимает список чисел (`measurement_results`).
2. Принимает от пользователя целое число N (делитель).
3. Подсчитывает, сколько элементов в списке `measurement_results` делятся нацело на N .

Исходные данные (Пример):

```
measurement_results = [12, 7, 25, 30, 15, 8, 40, 20]
```

Ввод пользователя (Пример): N=5

Технические Шаги:

1. Инициализация: Создать счетчик (count), установленный на 0.
2. Итерация: Использовать цикл for, чтобы пройти по каждому числу в списке.
3. Проверка кратности: Внутри цикла применить оператор остатка от деления (%). Если $A \% N = 0$, значит, число A кратно числу N.
4. Подсчет: Если условие истинно, увеличить счетчик на 1.

Пример Вывода (Для N=5):

```
--- Контроль Качества ---
Исходные результаты: [12, 7, 25, 30, 15, 8, 40, 20]
Проверяемый стандарт (делитель N): 5
-----
Результат 12: 12 % 5 = 2. Не соответствует.
Результат 7: 7 % 5 = 2. Не соответствует.
Результат 25: 25 % 5 = 0. Соответствует! (+1)
Результат 30: 30 % 5 = 0. Соответствует! (+1)
...
-----
```

Общее количество измерений, соответствующих стандарту (кратных 5): 5

Задача 4: Фильтрация Запросов по Длине (Удаление элементов с определённой длиной)

Вы — администратор поисковой системы. У вас есть список пользовательских запросов. Запросы, которые слишком короткие (меньше K символов), считаются нерелевантными или ошибочными. Вам нужно удалить все такие "спамные" запросы из основного списка.

Требуется:

Написать программу, которая:

1. Принимает список строк (user_queries).

2. Принимает от пользователя целое число K (минимальная допустимая длина).
3. Удаляет из исходного списка все строки, длина которых строго меньше K .

Исходные данные (Пример):

```
user_queries = ["купить", "авто", "телефон", "цены",  
"дом", "смартфон"]  
K = 5
```

Ожидаемый результат (Пояснение для $K=5$):

- "купить" (6) -> Оставляем
- "авто" (4) -> Удаляем
- "телефон" (7) -> Оставляем
- "цены" (4) -> Удаляем
- "дом" (3) -> Удаляем
- "смартфон" (8) -> Оставляем

Пример Вывода:

```
--- Фильтрация Поисковых Запросов ---  
Исходный список запросов: ['купить', 'авто', 'телефон',  
'цены', 'дом', 'смартфон']  
Минимальная длина (K): 5  
Запрос 'авто' (4 символа) удален.  
Запрос 'цены' (4 символа) удален.  
Запрос 'дом' (3 символа) удален.  
Финальный список релевантных запросов: ['купить',  
'телефон', 'смартфон']
```

Задача 5: Ротация Списка Задач (Сдвиг элементов)

Вы — менеджер проекта. У вас есть список задач на неделю. Из-за незапланированных событий все задачи должны быть сдвинуты вправо на K дней (позиций). При этом задачи, вытесненные с конца недели, должны появиться в начале (циклический сдвиг).

Требуется:

Написать программу, которая принимает список `task_list` и целое число `K` (количество позиций сдвига вправо). Сдвиг должен быть циклическим и корректно работать, даже если `K` больше длины списка.

Исходные данные (Пример):

```
task_list = ['Пн', 'Вт', 'Ср', 'Чт', 'Пт']  
K = 2
```

Ожидаемый результат (Пояснение):

Сдвиг на `K=2`:

- Пт, Чт (последние `K=2` элемента) перемещаются в начало.
- Исходный список ['Пн', 'Вт', 'Ср', 'Чт', 'Пт']
- Сдвиг -> ['Чт', 'Пт', 'Пн', 'Вт', 'Ср']

Технические Шаги:

1. Нормализация сдвига: Найти эффективное количество сдвигов (`K` эффективное) с помощью оператора остатка от деления (%). Если `K=7` при длине 5, то $7 \% 5 = 2$ - сдвиг эквивалентен 2 позициям.
 $K \text{ эффективное} = K \% \text{len(список)}$
2. Разделение: Разделить список на две части с помощью срезов:
 - Конец списка: `L_1 = список[:длина - K эффективное]`
(Элементы, которые уходят вправо)
 - Начало списка: `L_2 = список[длина - K эффективное:]`
(Элементы, которые придут слева)
3. Сборка: Объединить части в новом порядке: `L_2 + L_1`.

Пример Вывода:

```
--- Ротация Списка Задач ---  
Исходный список задач: ['Пн', 'Вт', 'Ср', 'Чт', 'Пт']  
Исходный сдвиг (K): 2  
Эффективный сдвиг: 2  
Результат: ['Чт', 'Пт', 'Пн', 'Вт', 'Ср']
```

Задача 6: Аудит Серийных Номеров (Обратный порядок для каждого второго элемента)

Вы проводите аудит базы данных серийных номеров. Из-за ошибки передачи, каждый второй серийный номер в списке записан в обратном порядке (зеркально). Ваша задача — исправить эту ошибку, провернув

каждый второй элемент списка обратно, чтобы восстановить его исходный вид.

Требуется:

Написать программу, которая принимает список `serial_numbers` и модифицирует его:

1. Используя цикл `for`, переберите список с шагом 2, начиная с индекса 1 (чтобы обрабатывать каждый второй элемент).
2. Для каждого такого элемента примените операцию переворота (например, используя срез `[::-1]`).
3. Замените исходный элемент в списке на его перевернутую версию.

Исходные данные (Пример):

```
serial_numbers = ['ABC100', '432ZYX', 'DEF200', '109WUT',  
'GHI300']  
# Элементы для исправления (индексы 1, 3): '432ZYX' и '109WUT'
```

Ожидаемый результат (Пояснение):

- Индекс 1: '432ZYX' -> 'XYZ234'
- Индекс 3: '109WUT' -> 'TUW901'
- Остальные элементы (0, 2, 4) остаются без изменений.

Пример Вывода:

```
--- Аудит Серийных Номеров ---  
Исходный список: ['ABC100', '432ZYX', 'DEF200', '109WUT',  
'GHI300']  
Исправлен элемент [1]: '432ZYX' -> 'XYZ234'  
Исправлен элемент [3]: '109WUT' -> 'TUW901'  
Финальный список: ['ABC100', 'XYZ234', 'DEF200',  
'TUW901', 'GHI300']
```

Задача 7: Сжатие Файла Лога (Удаление подряд идущих дубликатов)

Вы занимаетесь обработкой системных логов. В журнале часто возникают длинные цепочки одинаковых сообщений, которые засоряют файл (например, "Ожидание", "Ожидание", "Ожидание"). Ваша задача — сжать лог, удалив все подряд идущие дубликаты и оставив от каждой цепочки только одно первое сообщение.

Требуется:

Написать программу, которая принимает список `log_entries` и возвращает новый список, где из каждой последовательности одинаковых элементов остался только один.

Исходные данные (Пример):

```
log_entries = ['Загрузка', 'Загрузка', 'Ошибка',  
'Ошибка', 'Ошибка', 'Ожидание', 'Ожидание',  
'Загрузка']
```

Ожидаемый результат (Пояснение):

- 'Загрузка', 'Загрузка' -> Оставляем только первый 'Загрузка'
- 'Ошибка', 'Ошибка', 'Ошибка' -> Оставляем только первый 'Ошибка'
- 'Ожидание', 'Ожидание' -> Оставляем только первый 'Ожидание'

Пример Вывода:

```
--- Сжатие Системного Лога ---  
Исходный лог: ['Загрузка', 'Загрузка', 'Ошибка', 'Ошибка',  
'Ошибка', 'Ожидание', 'Ожидание', 'Загрузка']  
Сжатый лог: ['Загрузка', 'Ошибка', 'Ожидание', 'Загрузка']
```

Задача 8: Сортировка Финансовых Операций (Разделение списка на положительные и отрицательные элементы)

Вы работаете в финансовом отделе. У вас есть список недавних транзакций (`transactions`), где положительные числа обозначают доходы (поступления), а отрицательные числа — расходы (списания). Вам нужно разделить этот общий список на два отдельных списка для дальнейшего анализа.

Требуется:

Написать программу, которая принимает список чисел `transactions` и создает два новых списка:

1. `incomes` (доходы/положительные числа, включая ноль).
2. `expenses` (расходы/отрицательные числа).

Исходные данные (Пример):

```
transactions = [1500, -50, 200, 0, -120, 300, -75, 10]
```

Ожидаемый результат (Пояснение):

- Доходы (Положительные и Ноль): 1500, 200, 0, 300, 10
- Расходы (Отрицательные): -50, -120, -75

Пример Вывода:

```

--- Финансовый Аудит ---
Исходные транзакции: [1500, -50, 200, 0, -120, 300, -75, 10]
-----
Доходы (Incomes): [1500, 200, 0, 300, 10]
Расходы (Expenses): [-50, -120, -75]

```

Задача 9: Генерация Координатной Сетки (Создание матрицы с индексами)

Вы — разработчик карты для новой видеоигры. Для системы позиционирования вам нужно создать координатную сетку $N \times M$. Каждая ячейка в этой сетке должна содержать свои точные координаты: [индекс строки (y), индекс столбца (x)].

Требуется:

Написать программу, которая:

1. Задаёт размер матрицы: N (количество строк) и M (количество столбцов).
2. Создает двумерный список (`coordinate_grid`) размера $N \times M$.
3. Каждый элемент матрицы должен быть списком из двух чисел — [индекс_строки, индекс_столбца].

Исходные данные (Пример):

N (строки) = 3

M (столбцы) = 4

Ожидаемый результат (Пояснение):

```

| [0, 0] | [0, 1] | [0, 2] | [0, 3] |
| [1, 0] | [1, 1] | [1, 2] | [1, 3] |
| [2, 0] | [2, 1] | [2, 2] | [2, 3] |

```

Пример Вывода:

```

--- Генерация Координатной Сетки 3x4 ---
[ [0, 0], [0, 1], [0, 2], [0, 3] ]
[ [1, 0], [1, 1], [1, 2], [1, 3] ]

```

```
[ [2, 0], [2, 1], [2, 2], [2, 3] ]
```